Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Spring, Year:2026), B.Sc. in CSE (Day)

Lab Report NO: 04

Course Title: Artificial Intelligence Lab

Course Code: CSE 316 **Section:** 232_D7

Lab Experiment Name: Alpha Beta Pruning

Student Details

Name	ID
MD:AL-Mamun	232002264

Lab Date : 16-04-2026

Submission Date : 09-04-2026

Course Teacher's Name : Md. Sabbir Hosen Mamun

Lab Report Status

Marks: …..... Date: …….. Signature: …….…….

Comments: ……..….....

1. TITLE OF THE LAB REPORT

Problem Solving by Minimax & Alpha-Beta Pruning.

2. OBJECTIVES

- To develop an implementation of the Minimax algorithm for decision-making in a binary game tree.
- To apply Alpha-Beta Pruning as an improved and optimized form of the Minimax algorithm.
- To construct a binary game tree with randomly generated leaf node values.
- To analyze and compare how many nodes are explored by both Minimax and Alpha-Beta Pruning.
- To determine the number of nodes that are pruned during the Alpha-Beta Pruning process.
- To evaluate the percentage improvement in efficiency achieved by Alpha-Beta Pruning over the standard Minimax algorithm.

3. PROCEDURE

Step 1: Import Required Libraries

- Import math for depth calculation
- Import random for generating random leaf node values

Step 2: Generate Leaf Nodes

- Decide number of leaf nodes
- Generate random integer values for each leaf node
- Store them in a list called scores

Step 3: Calculate Tree Depth

- Use the formula:
- treeDepth = log2(number of leaf nodes)
- This determines how deep the binary tree will be

Step 4: Implement Minimax Algorithm

- Start from root node
- Recursively explore left and right child nodes
- Apply:
- MAX level → take maximum value
- MIN level → take minimum value
- Continue until reaching leaf nodes
- Return the optimal value

Step 5: Count Nodes in Minimax

- Maintain a counter variable
- Increase counter every time a node is visited
- This gives total nodes evaluated by Minimax

Step 6: Implement Alpha-Beta Pruning

- Start from root node
- Use two values:
- alpha (best value for MAX)
- beta (best value for MIN)
- Recursively evaluate nodes like Minimax
- If condition beta ≤ alpha occurs:
- Stop exploring remaining branch (prune it)

Step 7: Count Nodes and Pruned Nodes

- Count total visited nodes in Alpha-Beta
- Count pruned nodes using:
- pruned = Minimax nodes − Alpha-Beta nodes

Step 8: Compute Efficiency Improvement

- Use formula:
- Improvement % = ((Minimax − AlphaBeta) / Minimax) × 100

Step 9: Display Output

- Print:
- Generated leaf nodes
- Minimax result and nodes evaluated
- Alpha-Beta result, nodes evaluated, and pruned nodes
- Efficiency improvement percentage

Step 10: Compare Results

- Verify that both algorithms give the same optimal value
- Confirm that Alpha-Beta uses fewer nodes than Minimax
- Conclude efficiency gain

4. IMPLEMENTATION

Minimax Function

```
def minimax(depth, index, isMax, scores, targetDepth, counter):
```

Purpose:

The Minimax function is used to find the optimal value in a game tree by exploring all possible nodes.

Working:

- If it is a MAX level, it selects the maximum value from child nodes.
- If it is a MIN level, it selects the minimum value from child nodes.
- It recursively evaluates all nodes until reaching the leaf level.

Alpha-Beta Pruning Function

```
def alpha_beta(depth, index, isMax, scores, targetDepth, alpha, beta, counter):
```

Purpose:
This is an optimized version of Minimax that reduces the number of nodes evaluated.

Working:

- Uses two values:
- Alpha (best value for MAX)
- Beta (best value for MIN)
- If beta ≤ alpha, further exploration of that branch is stopped (pruning).

Leaf Node Generation

```
scores = list(map(int, input("Enter leaf nodes: ").split()))
```

Purpose:
Generates random values for leaf nodes of the game tree.

Explanation:
These values represent the terminal states of the game.

Tree Depth Calculation

```
treeDepth = int(math.log2(n))
```

Purpose:
Calculates the depth of the binary tree.

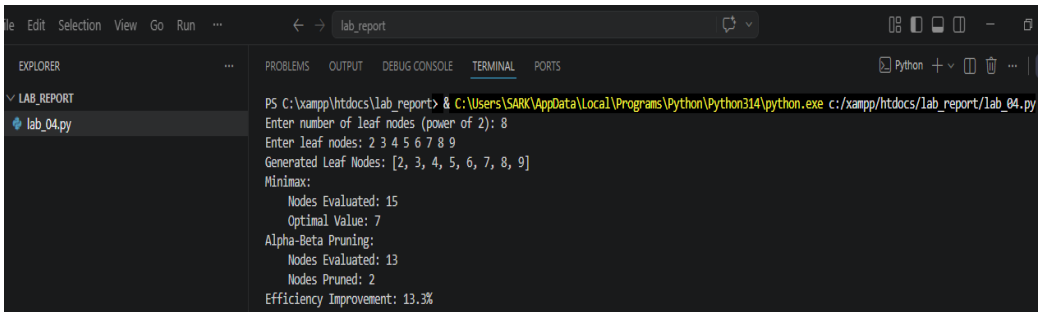
Efficiency Calculation

```
improvement = ((minimax_counter["evaluated"] - ab_counter["evaluated"]) /
 minimax_counter["evaluated"]) * 100
```

Purpose:
Calculates how much more efficient Alpha-Beta pruning is compared to Minimax.

5. TEST RESULT

Case 1:



In this case, the program generated a set of random leaf node values and constructed a binary game tree. The Minimax algorithm evaluated all nodes in the tree to determine the optimal value. In contrast, the Alpha-Beta Pruning algorithm skipped several unnecessary branches, resulting in fewer node evaluations. The optimal value obtained from both algorithms is the same, but Alpha-Beta Pruning achieved better efficiency by reducing the number of evaluated nodes and pruning some branches.

Case 2:

```
PS C:\xampp\htdocs\lab_report> & C:\Users\SARK\AppData\Local\Programs\Python\Python314\python.exe c:/xampp/htdocs/lab_report/lab_04.py
Enter number of leaf nodes (power of 2): 8
Enter leaf nodes: 11 2 3 13 4 7 9 15
Generated Leaf Nodes: [11, 2, 3, 13, 4, 7, 9, 15]
Minimax:
    Nodes Evaluated: 15
    Optimal Value: 11
Alpha-Beta Pruning:
    Nodes Evaluated: 12
    Nodes Pruned: 4
Efficiency Improvement: 20.0%
PS C:\xampp\htdocs\lab_report>
```

In this case, a different set of random leaf node values was generated. The Minimax algorithm again explored the entire tree, evaluating all nodes. However, Alpha-Beta Pruning reduced the number of explored nodes by pruning branches that did not affect the final result. This led to improved efficiency while still producing the same optimal value as Minimax.

6. DISCUSSION

This experiment is centered on the design and implementation of the Minimax algorithm along with Alpha-Beta Pruning using Python on a binary game tree structure.

The main goals of the lab were accomplished by generating random values at the leaf nodes, executing both algorithms, and analyzing their performance differences.

A systematic approach was followed throughout the implementation. This included creating leaf nodes, determining the depth of the tree, performing recursive evaluation using the Minimax algorithm, and enhancing the process through Alpha-Beta Pruning for better efficiency.

The program effectively applies recursion and decision-making conditions to evaluate each node. It also incorporates pruning techniques to skip unnecessary branches when certain conditions are met, reducing computation.

From Case 1 and Case 2, it is observed that both algorithms return the same optimal result. However, Alpha-Beta Pruning reduces the number of evaluated nodes, thereby improving overall efficiency.

Overall, the experiment clearly highlights how optimization techniques like Alpha-Beta Pruning can enhance game tree search performance, though the level of improvement can depend on how the leaf node values are arranged.

7. Conclusion

This experiment effectively illustrated the implementation of both the Minimax algorithm and Alpha-Beta Pruning on a binary game tree. The results confirmed that both approaches yield the same optimal outcome, ensuring the correctness of the implementation.

However, Alpha-Beta Pruning proved to be more efficient by reducing the number of nodes that needed to be evaluated. This optimization leads to faster performance compared to the standard Minimax approach.

Overall, the findings emphasize the significance of using optimization techniques in decision-making processes, particularly in large and complex game trees where computational efficiency becomes crucial.

For source code and documentation are given below

GitHub Link: https://github.com/abdullahalmamun161666/AI_lab_report.git